

Cours de Bases de Données orientées Web

Line Jakubiec-Jamet (BD)
Aïtor Gonzales (Web)

L3 Informatique et Mathématiques-Informatique (MI et DLMI)

January 18, 2026

Introduction aux bases de données

Algèbre relationnelle

Le langage SQL

Introduction

- ▶ **Base de données** : ensemble organisé et structuré d'informations
- ▶ **But** : stockage et exploitation de grandes quantités d'informations
- ▶ **Enjeu** : place importante dans le domaine de l'informatique de gestion
- ▶ **Méthodes** : développement de concepts et d'algorithmes spécifiques
 - stockage (modèles et opérations)
 - mémoire secondaire (et non RAM)
 - Web (volume de données toujours croissant)

Quelques définitions

- ▶ **Donnée** : code convenu d'un objet, d'un phénomène, d'une image, d'un son, d'un texte...
- ▶ **Information** : signification que l'on attache à un ensemble de données.
→ Exemple : la météo dans un journal
(température, état du ciel et froid)
- ▶ **Banque de données** : vaste ensemble de données faiblement structuré.
→ Exemple : une liste de livres avec leur code
- ▶ **Schéma de données** : résultat de la modélisation d'un univers donné.
→ Exemple : l'organigramme d'une entreprise

Systèmes de Gestion de Bases de Données (SGBD)

C'est un ensemble de programmes qui permet de gérer et de manipuler une ou plusieurs bases de données. Ces systèmes ont plusieurs objectifs :

- ▶ **indépendance physique** : le niveau utilisateur doit être transparent à un changement des structures de stockage utilisées.
- ▶ **indépendance logique** : le niveau utilisateur doit être transparent à un changement de l'organisation des données.
- ▶ **sécurité (intégrité)** : les données doivent respecter les contraintes du monde réel modélisé.
- ▶ **sécurité (confidentialité)** : tout utilisateur n'a pas obligatoirement accès à toutes les données.

Systèmes de Gestion de Bases de Données (suite)

- ▶ **disponibilité (gestion de la concurrence)** : tout utilisateur doit pouvoir ignorer l'existence d'autres utilisateurs.
- ▶ **disponibilité (performances)** : l'accès aux données doit être optimisé.
- ▶ **non redondance** : chaque donnée ne doit être présente qu'une seule fois dans la base (mise à jour).
- ▶ **cohérence** : les données sont soumises à un certain nombre de contraintes d'intégrité qui définissent un état.
- ▶ **partage** : dans un contexte multi-utilisateurs, il faut permettre à plusieurs utilisateurs de modifier la même donnée en même temps et assurer un résultat cohérent de la base.
- ▶ **résistance aux pannes** : les mécanismes de reprise sur panne doivent assurer la cohérence des données.

Exemples de Systèmes de Gestion de Bases de Données (SGBD)

- ▶ PostgreSQL : *http : //www.postgresql.org/* (dans le domaine public)
- ▶ MySQL : *http : //www.mysql.org/* (dans le domaine public)
- ▶ Oracle : *http : //www.oracle.com/* (de Oracle Corporation)
- ▶ IBM DB2 :
http : //www – 306.ibm.com/software/data/db2/
- ▶ Microsoft SQL : *http : //www.microsoft.com/sql/*
- ▶ Sybase : *http : //www.sybase.com/linux*
- ▶ Informix :
http : //www – 306.ibm.com/software/data/informix/

Exemples de bases de données

- ▶ Gestion d'une bibliothèque (livres, abonnés, emprunts, réservations...)
- ▶ Gestion d'une session d'examens d'université (étudiants, promotions, salles, dates, emplois du temps...)
- ▶ Gestion des clients d'un magasin (achats, retour marchandises, stock...)
- ▶ Gestion d'une agence immobilière (appartements, visites, ventes, locations...)
- ▶ Gestion d'une compagnie d'assurances (sinistres, sociétaires, produits assurés...)
- ▶ Applications en ligne (site Web spécifique)

Introduction à l'algèbre relationnelle

Support mathématique qui définit une théorie permettant la manipulation des données.

- ▶ **attribut** : identificateur décrivant une information stockée (ex : nom_etudiant).
- ▶ **domaine** : ensemble, fini ou infini, des valeurs possibles d'un attribut (ex : Dupont).

Introduction à l'algèbre relationnelle (suite)

- ▶ **relation** : ensemble de n attributs ($n > 0$).

ex : ETUDIANT(no_etudiant, nom_etudiant, prenom_etudiant,
datnaiss_etudiant)

no_etudiant	nom_etudiant	prenom_etudiant	datnaiss_etudiant
-------------	--------------	-----------------	-------------------

- ▶ **schéma** : relation dans laquelle est précisée la liste des attributs avec leurs domaines.

ex :

no_etudiant	nom_etudiant	prenom_etudiant	datnaiss_etudiant
11628815	Dupont	Jean	25/04/1985
11688810	Dubois	Marie	07/08/1984
12099515	Durand	Bastien	20/01/1984
12331234	Delaporte	Ingrid	16/05/1985

Les opérateurs relationnels

But : décrire les opérations qu'il est possible d'appliquer sur les relations et les schémas.

Trois catégories d'opérateurs relationnels :

- ▶ Les **opérateurs unaires** (Sélection, Projection) : pour produire une nouvelle relation à partir d'une autre relation.
- ▶ Les **opérateurs binaires ensemblistes** (Union, Intersection, Différence) : pour produire une nouvelle relation à partir de deux relations de mêmes degré et domaine.
- ▶ Les **opérateurs binaires ou n -aires** (Produit cartésien, Jointure, Division) : pour produire une nouvelle relation à partir de deux ou plusieurs autres relations.

L'opérateur de sélection

La sélection sur une relation $R(X)$ suivant une condition de sélection E est définie par :

$$R : E = \{L \in A(X) \text{ tq } (L \in R) \text{ et } (E(L) = \text{vrai})\}$$

ex :

no_etudiant	nom_etudiant	prenom_etudiant	datnaiss_etudiant
11628815	Dupont	Jean	25/04/1985
11688810	Dubois	Marie	07/08/1984
12099515	Durand	Bastien	20/01/1984
12331234	Delaporte	Ingrid	16/05/1984

Si $E = (\text{datnaiss_etudiant} \geq 01/07/1984)$ alors $R : E$ est telle que :

no_etudiant	nom_etudiant	prenom_etudiant	datnaiss_etudiant
11628815	Dupont	Jean	25/04/1985
11688810	Dubois	Marie	07/08/1984

L'opérateur de projection

La projection sur Y ($Y \subseteq X$) d'une relation $R(X)$ est la relation $R[Y]$ définie par :

$$R[Y] = \{L \in A(Y) \text{ tq } \exists L' \in A(X), (L'/Y = L) \text{ et } (L' \in R)\}$$

ex :

no_etudiant	nom_etudiant	prenom_etudiant	datnaiss_etudiant
11628815	Dupont	Jean	25/04/1985
11688810	Dubois	Marie	07/08/1984
12099515	Durand	Marie	20/01/1984

Soit $Y = \{prenom_etudiant\}$, alors $R[Y]$ est telle que :

prenom_etudiant
Jean
Marie

L'opérateur union

L'union de $R(X)$ et de $S(X)$ est la relation $(R \cup S)(X)$ définie par :

$$R \cup S = R + S$$

ex : Soient deux relations R et S :

no_etudiant	nom_etudiant	prenom_etudiant	datnaiss_etudiant
11628815	Dupont	Jean	25/04/1985
11688810	Dubois	Marie	07/08/1984

no_etudiant	nom_etudiant	prenom_etudiant	datnaiss_etudiant
12099515	Durand	Bastien	20/01/1984
12331234	Delaporte	Ingrid	16/05/1985

L'opérateur union (suite)

Alors $(R \cup S)$ est telle que :

no_etudiant	nom_etudiant	prenom_etudiant	datnaiss_etudiant
11628815	Dupont	Jean	25/04/1985
11688810	Dubois	Marie	07/08/1984
12099515	Durand	Bastien	20/01/1984
12331234	Delaporte	Ingrid	16/05/1985

L'opérateur intersection

L'intersection de $R(X)$ et de $S(X)$ est la relation $(R \cap S)(X)$ définie par :

$$R \cap S = R * S$$

ex : Soient deux relations R et S :

no_etudiant	nom_etudiant	prenom_etudiant	datnaiss_etudiant
11628815	Dupont	Jean	25/04/1985
11688810	Dubois	Marie	07/08/1984

no_etudiant	nom_etudiant	prenom_etudiant	datnaiss_etudiant
12099515	Durand	Bastien	20/01/1984
11628815	Dupont	Jean	25/04/1985
12331234	Delaporte	Ingrid	16/05/1985

L'opérateur intersection(suite)

Alors $(R \cap S)$ est telle que :

no_etudiant	nom_etudiant	prenom_etudiant	datnaiss_etudiant
11628815	Dupont	Jean	25/04/1985

L'opérateur différence

La différence de $R(X)$ et $S(X)$ est définie par :

$$R - S = \{L \in A(X) \text{ tq } (L \in R) \text{ et } (L \notin S)\} \text{ ou bien } R - S = R * \neg S$$

ex : Soient deux relations R et S :

no_etudiant	nom_etudiant	prenom_etudiant	datnaiss_etudiant
11628815	Dupont	Jean	25/04/1985
11688810	Dubois	Marie	07/08/1984
12099515	Durand	Bastien	20/01/1984

no_etudiant	nom_etudiant	prenom_etudiant	datnaiss_etudiant
12099515	Durand	Bastien	20/01/1984
11628815	Dupont	Jean	25/04/1985
12331234	Delaporte	Ingrid	16/05/1985

L'opérateur différence (suite)

Alors $(R - S)$ est telle que :

no_etudiant	nom_etudiant	prenom_etudiant	datnaiss_etudiant
11688810	Dubois	Marie	07/08/1984

L'opérateur produit cartésien

Le produit cartésien de $R(X)$ et $S(Y)$ est défini par :

$$R \times S = R * S$$

ex : Soient deux relations R et S :

no_etudiant	nom_etudiant
11628815	Dupont
11688810	Dubois
12099515	Durand

ue_info	ue_bio
Programmation	Génomique
Bases de données	Biochimie

L'opérateur produit cartésien (suite)

Alors ($R \times S$) est telle que :

no_etudiant	nom_etudiant	ue.info	ue.bio
11628815	Dupont	Programmation	Génomique
11628815	Dupont	Bases de données	Biochimie
11688810	Dubois	Programmation	Génomique
11688810	Dubois	Bases de données	Biochimie
12099515	Durand	Programmation	Génomique
12099515	Durand	Bases de données	Biochimie

L'opérateur jointure (ou jointure naturelle)

La jointure de $R(X)$ et $S(Y)$ est définie par :

$$Z = X \cup Y$$

$$R * S = \{L \in A(Z) \text{ tq } (L/X \in R) \text{ et } (L/Y \in S)\}$$

ex : Soient deux relations R et S :

no_etudiant	nom_etudiant
11628815	Dupont
11688810	Dubois
12099515	Durand

no_etudiant	option
11628815	Programmation
11688810	Biochimie
12099515	Biochimie

L'opérateur jointure (suite)

Alors $(R * S)$ est telle que :

no_etudiant	nom_etudiant	ue_info
11628815	Dupont	Programmation
11688810	Dubois	Biochimie
12099515	Durand	Biochimie

L'opérateur division

La division de $R(X)$ et $S(Y)$ est définie par :

$$Z = X - Y$$

$$R : S = \{L \in A(Z) \text{ tq}$$

$$\forall L' \in A(X), \text{ si } (L'/Z = L) \text{ et } (L'/Y \in S) \text{ alors } L' \in R\}$$

ex : Soient deux relations R et S :

nom_etudiant	UE
Dupont	Programmation
Dupont	Biochimie
Dupont	Analyse
Dubois	Biochimie
Durand	Programmation
Durand	Biochimie
Delaporte	Biochimie

L'opérateur division (suite)

UE
Programmation
Biochimie

Alors $(R : S)$ est telle que :

nom_etudiant
Dupont
Durand

Présentation de SQL

- ▶ **But** : accéder et gérer les données et les informations caractérisant les bases de données
- ▶ **SQL** : Structured Query Language
- ▶ **Définition** : Langage d'accès normalisé aux bases de données
 - traitement d'un ensemble d'enregistrements
 - chemin d'accès aux données défini automatiquement
- ▶ **Mais aussi** : Langage d'accès normalisé à l'information contenue dans les bases de données
 - traitement d'un ensemble de valeurs effectives
 - chemin d'accès aux informations défini automatiquement
- ▶ **Exemples de systèmes SQL** : Oracle, SQLite, SQL-Server (Microsoft) et MySQL, PostgreSQL, MariaDB

Présentation de SQL (suite)

► Trois catégories d'instructions :

- les instructions de définition de données (**DDL**)
 - gestion des relations (tables) avec leurs attributs
 - rôle du concepteur de bases de données
- les instructions de contrôle de données (**DCL**)
 - gestion de la politique d'accès
 - rôle de l'administrateur de la base de données
- les instructions de manipulation de données (**DML**)
 - gestion et organisation des données en vue de leur exploitation
 - rôle du développeur de la base de données

Les instructions de définition de données

⇒ Elles agissent sur la structure de la base de données.

- ▶ créer, modifier, supprimer des relations (tables)
- ▶ définir le domaine des données (ex : nombre, chaîne de caractères, date, booléen)
- ▶ ajouter des contraintes sur les données (ex : non nullité d'un attribut, clé primaire, compatibilité entre des dates)

⇒ Instructions utiles :

CREATE, ALTER, DROP, RENAME, TRUNCATE

Les instructions de contrôle de données

⇒ Elles gèrent les droits d'accès aux tables :

- ▶ GRANT : attribue un privilège à un utilisateur
(ex : GRANT SELECT ON table TO utilisateur).
- ▶ REVOKE : supprime un privilège à un utilisateur
(ex : REVOKE SELECT ON table TO utilisateur).

⇒ Elles gèrent les transactions, la validation et l'annulation des modifications de données :

- ▶ COMMIT : valide la transaction en cours.
- ▶ SAVEPOINT : sauvegarde de la transaction en un certain point (restauration).
- ▶ ROLLBACK : annule la transaction en cours.

Les instructions de manipulation de données

⇒ Ce sont les requêtes

- ▶ pour interroger les bases de données (extraction et traitement des données)
- ▶ **Lien direct avec l'algèbre relationnelle :**
 - sélectionner certaines colonnes d'une table (projection)
 - sélectionner certaines lignes d'une table en fonction de leur contenu (sélection)
 - combiner des informations venant de plusieurs tables (jointure, union, intersection, différence et division)
 - combiner entre elles ces différentes opérations.

⇒ Instructions utiles :

INSERT, UPDATE, DELETE, SELECT

L'instruction SELECT : première approche

⇒ **Idée** : la base de données existe déjà et contient des informations; on l'interroge.

⇒ **Syntaxe générale** :

```
SELECT [ ALL | DISTINCT ] { * | attribut [, ...] }  
      FROM nom_table [, ...]  
      [ WHERE condition ]
```

- ▶ éléments obligatoires et optionnels
- ▶ choix, alternatives
- ▶ listes de longueur variable

L'instruction SELECT : première approche (suite)

⇒ **Exemples :**

```
SELECT * FROM table;
```

```
SELECT a_1, a_2 FROM table;
```

```
SELECT DISTINCT a_1 FROM table WHERE a_1 > 0;
```

```
SELECT a_1, a_2 FROM table WHERE a_1 = a_2;
```

```
SELECT a_1+a_2 FROM table;
```

```
SELECT a_1+a_2 somme FROM table;
```

```
SELECT a_1, a_2 FROM table
```

```
    WHERE (a_1+a_2)*50 < 1000;
```

```
SELECT x_0, y_0 FROM table1, table2
```

```
    WHERE table1.x_0 = table2.y_0;
```


L'instruction SELECT : dans le détail

⇒ **Idée** : la requête SELECT peut être affinée selon des critères de recherche plus précis.

- ▶ expression des conditions (dans le WHERE)
- ▶ organisation de l'affichage et du traitement des données (tri)
- ▶ utilisation de fonctions SQL spécifiques (calcul, conversion)
- ▶ traitement des dates (algèbre spécifique)
- ▶ utilisation de sous-requêtes (diviser pour régner!)
- ▶ manipulation des opérateurs ensemblistes (intersection, union, différence)
- ▶ traitement de l'absence de valeur (jointure)

L'instruction SELECT : les conditions (dans le WHERE)

- ▶ Opérateurs de comparaison : `=`, `<>`, `!=`, `>`, `>=`, `<`, `<=`
- ▶ Test de valeurs nulles : **IS NULL**
- ▶ Test d'appartenance (ou non-appartenance) d'une valeur d'un attribut à une liste : **IN** (ou **NOT IN**)
- ▶ Intervalle : **BETWEEN ... AND ...**
- ▶ Expressions génériques : **LIKE** % et _ (ou **NOT LIKE** % et _)
- ▶ Opérateurs logiques : **AND** et **OR**

L'instruction SELECT : organisation des données (tri)

```
SELECT [ ALL | DISTINCT ] { * | attribut [, ...] }  
      FROM nom_table [, ...]  
      [ WHERE condition ]  
      [ GROUP BY attribut [, ...]  
      [ HAVING condition_groupe]]  
      [ ORDER BY attribut [, ...]]
```

- ▶ **GROUP BY** : définition des groupes (sous-ensembles, agrégats).
- ▶ **HAVING** : spécification d'un filtre (condition de regroupement des agrégats)
- ▶ **ORDER BY** : tri des agrégats du résultat

L'instruction SELECT : tri par ORDER BY

```
SELECT [ ALL | DISTINCT ] { * | attribut [, ...] }  
      FROM nom_table [, ...]  
      [ WHERE condition ]  
      [ ORDER BY expression [ ASC | DESC ] [, ...]]
```

- ▶ *expression* désigne soit une colonne, soit une opération mathématique de base sur les colonnes.
- ▶ **ASC** : ordre ascendant du tri (par défaut)
- ▶ **DESC** : ordre descendant du tri
- ▶ si *expression* est répété plusieurs fois, le tri se fait dans l'ordre d'apparition des expressions et selon les précédentes.
- ▶ **ORDER BY** : opère toujours sur le résultat final de la requête.

L'instruction SELECT : fonctions spécifiques (calcul sur agrégats)

- ▶ **COUNT(*)** : comptage des lignes
- ▶ **COUNT([DISTINCT | ALL] expression)** : comptage des valeurs
- ▶ **MAX([DISTINCT | ALL] expression)** : maximum des valeurs
- ▶ **MIN([DISTINCT | ALL] expression)** : minimum des valeurs
- ▶ **SUM([DISTINCT | ALL] expression)** : somme des valeurs
- ▶ **AVG([DISTINCT | ALL] expression)** : moyenne des valeurs
- ▶ **STDDEV([DISTINCT | ALL] expression)** : écart-type des valeurs
- ▶ **VARIANCE([DISTINCT | ALL] expression)** : variance des valeurs

Quelques fonctions SQL : les fonctions numériques

⇒ les nombres sont typés
INTEGER, INT, NUMBER, NUMERIC, DECIMAL, DEC

- ▶ **SIG(nb)** : signe du nombre *nb*
- ▶ **ABS(nb)** : valeur absolue de *nb*
- ▶ **COS(nb)** : cosinus de *nb*
- ▶ **SIN(nb)** : sinus de *nb*
- ▶ **TAN(nb)** : tangente de *nb*
- ▶ **LN(nb)** : logarithme naturel de *nb*
- ▶ **LOG(base,nb)** : logarithme en base quelconque
- ▶ **SQRT(nb)** : racine carrée de *nb*
- ▶ **POWER(nb,puissance)** : puissance de *nb*

Quelques fonctions SQL : les fonctions numériques (suite)

- ▶ **ROUND(nb [,position])** : arrondi à la position spécifiée ;
un entier positif (resp. négatif) indique une position après
(resp. avant) la virgule. Par défaut, arrondi à l'unité.
- ▶ **TRUNC(nb [,position])** : troncature à la position spécifiée;
un entier positif (resp. négatif) indique une position après
(resp. avant) la virgule. Par défaut, arrondi à l'unité.
- ▶ **TO_CHAR(nb [,format])** : conversion d'un nombre en chaîne.
format est une chaîne formée par exemple des éléments suivants :
 - 0 : un chiffre ou un 0 si absence de chiffres
 - S : signe du nombre
 - RM : écriture en chiffres romains
 - EEEE : écriture scientifique d'un nombre

Quelques fonctions SQL : les fonctions sur les chaînes de caractères

- ⇒ les chaînes de caractères sont typées
VARCHAR, VARCHAR2, TEXT, STRING
- ▶ **LENGTH(chaîne)** : longueur de la chaîne en caractères
 - ▶ **LENGTHB(chaîne)** : longueur de la chaîne en octets
 - ▶ **ASCII(chaîne)** : code décimal du premier caractère
 - ▶ **TO_NUMBER(chaîne)** : nombre correspondant à la chaîne
 - ▶ **LOWER(chaîne)** : met la chaîne en lettres minuscules
 - ▶ **UPPER(chaîne)** : met la chaîne en lettres majuscules
 - ▶ **INITCAP(chaîne)** : met en majuscules les premières lettres de chaque mot
 - ▶ **SOUNDEX(chaîne)** : retourne la chaîne phonétique

Quelques fonctions SQL : les fonctions sur les chaînes de caractères (suite 1)

- ▶ **INSTR(chaîne, sous_chaîne [,position [,n]])** :
A partir d'une position (par défaut 1), exprimée en nombre de caractères (position relative au début de la chaîne si *position* est positif, ou relative à sa fin si *position* est négatif), recherche la position de la n^{eme} occurrence de la sous-chaîne (par défaut la première occurrence). Retourne 0 si la recherche échoue.
- ▶ **SUBSTR(chaîne, position [,longueur])** :
A partir d'une position, extrait une sous-chaîne de longueur donnée (par défaut jusqu'à la fin de la chaîne).
- ▶ **REPLACE(chaîne, sous_chaîne [,sous_chaîne_de_remplacement])** :
Remplace ou supprime toutes les occurrences d'une sous_chaîne.

Quelques fonctions SQL : les fonctions sur les chaînes de caractères (suite 2)

- ▶ **LPAD(chaîne, longueur, [,chaîne_répétée]) :**
RPAD(chaîne, longueur, [,chaîne_répétée]) :
Répétition, à gauche (ou à droite) de la chaîne, d'une autre chaîne (par défaut un blanc) jusqu'à obtenir la longueur spécifiée.
- ▶ **LTRIM(chaîne [,ensemble_de_caractères_à_supprimer]) :**
RTRIM(chaîne [,ensemble_de_caractères_à_supprimer]) :
Supprime à gauche (ou à droite) de la chaîne, les occurrences des caractères à supprimer (par défaut un blanc).

SQL et le traitement des dates

⇒ Les dates sont typées DATE

ex : 31-01-2013 21 :54 :45 (31/01/2013 21 :54 :45 ...)

⇒ *SYSDATE* retourne la date courante (système).

- ▶ **LAST_DAY(date)** : dernière date du mois de la date passée en argument.
- ▶ **NEXT_DAY(date)** : première date postérieure à la date passée en argument et qui correspond au jour de la semaine passé en argument.
- ▶ **ADD_MONTHS(date,nombre_mois)** : date passée en argument augmentée d'un nombre de mois.
- ▶ **MONTHS_BETWEEN(date1,date2)** : nombre de mois qui séparent *date1* et *date2*.
- ▶ **NEW_TIME(date,timezone1,timezone2)** : date passée en argument convertie par changement de zone horaire (ex : GMT, AST).

SQL et le traitement des dates (suite)

- ▶ **TO_CHAR**(date [,format [,nls_langue]]) : conversion d'une date en une chaîne de caractères.
- ▶ **TO_DATE**(chaîne [,format [,nls_langue]]) : conversion d'une chaîne de caractères en date.

- **Exemples pour *nls_langue*** : french, american, arabic, german

- **Exemples pour *format*** :

CC (siècle), *YEAR* (année en lettres), *YYYY* (année sur 4 chiffres),
YY (année sur 2 chiffres), *Q* (numéro du trimestre), *MONTH* (mois en lettres),
MM (mois), *WW* (numéro de la semaine dans l'année), *DAY* (jour en lettres),
DDD (numéro du jour dans l'année), *DD* (numéro du jour dans le mois),
D (numéro du jour dans la semaine), *HH* (heure sur 12 heures),
HH24 (heure sur 24 heures), *MI* (minutes), *SS* (secondes par minute)

Les sous-requêtes SQL

- ⇒ attention au **nombre de lignes** retournées par la sous-requête.
 - ⇒ attention aux **types** utilisés.
 - ⇒ attention au **sens** donné aux sous-requêtes.
- 1) Tous les opérateurs vus dans le cours peuvent être utiles à l'expression des sous-requêtes (conditions, listes).
 - 2) On peut écrire des sous-requêtes d'existence.
 - 3) On peut écrire des sous-requêtes en clause *HAVING*.

Les sous-requêtes SQL (suite 1)

Sous-requêtes ramenant au plus une ligne

```
SELECT... FROM ...  
WHERE exp op (SELECT exp FROM ...);
```

avec $op \in \{=, \neq, <, >, \leq, \geq\}$

Il y a une erreur d'exécution si la requête ramène plusieurs lignes.

```
SELECT... FROM ...  
WHERE (exp1,...,expN) op (SELECT exp1,...,expN FROM ...);
```

avec $op \in \{=, \neq, <>\}$

Les sous-requêtes SQL (suite 2)

Sous-requêtes ramenant 0, 1 ou plusieurs lignes

```
SELECT... FROM ...  
WHERE exp op (SELECT exp FROM ...);
```

avec $op \in \{IN, NOT IN\}$

Les sous-requêtes SQL (suite 3)

Sous-requêtes d'existence

```
SELECT... FROM ...  
WHERE [NOT] EXISTS (SELECT exp FROM ...);
```

Dans la sous-requête, pour faire référence à une colonne extérieure qui a le même nom qu'une colonne intérieure, il faut la préfixer par le nom de la table dont elle provient.

Les sous-requêtes SQL (suite 4)

Sous-requêtes en clause HAVING

SELECT... FROM ...

WHERE ...

GROUP BY ...

HAVING exp1 op exp2;

avec $op \in \{=, \neq, <, >, \leq, \geq\}$

SQL et la manipulation des opérateurs ensemblistes

⇒ Au plus près de l'algèbre relationnelle

- ▶ L'opérateur **union** : $relation1 \cup relation2$
SELECT * FROM relation_1 UNION SELECT * FROM relation_2;
- ▶ L'opérateur **intersection** : $relation1 \cap relation2$
SELECT * FROM relation_1 INTERSECT SELECT * FROM relation_2;
- ▶ L'opérateur **différence** : $relation1 - relation2$
SELECT * FROM relation_1 EXCEPT SELECT * FROM relation_2;

Recherche de l'absence de valeur

```
SELECT ... FROM ...  
WHERE exp IS [NOT] NULL;
```

et non :

```
SELECT ... FROM ...  
WHERE exp [= | <>] NULL;
```

car toute opération de comparaison qui comporte une expression qui n'a pas de valeurs, prend la valeur INDEFINI et n'est donc pas vraie.

Calculs avec des absences de valeur

- ▶ Un calcul exprimé avec des opérateurs arithmétiques n'a pas de valeurs lorsqu'**au moins une** de ses composantes n'a pas de valeurs.
- ▶ Un calcul de chaîne exprimé avec l'opérateur `||` n'a pas de valeurs lorsque **toutes** ses composantes n'ont pas de valeurs.

Valeur par défaut de l'absence de valeur

⇒ la fonction spécifique *NVL*

$NVL(exp1, exp2) = exp1$ si $exp1$ est définie

$NVL(exp1, exp2) = exp2$ sinon

ex : pour qu'un attribut indéfini (inconnu dans la table) soit considéré comme une valeur nulle (pourvu que cet attribut soit un nombre), on écrit :

Select NVL(attribut,0) from table;

Le cas de la jointure entre deux tables en cas d'absence de valeur (jointure externe)

- ▶ C'est une opération entre deux tables :
une **table externe** identifiée par (+) et une **table de base**.
- ▶ **Résultat de cette jointure** : lignes obtenues par
la jointure normale des deux tables, augmentées des lignes
de la table de base qui n'y figurent pas (il y a alors
absence de valeur sur les colonnes de la table externe).

```
SELECT ... FROM table1,table2,...  
WHERE table1.attribut1 op table2.attribut2(+)  
[AND table1.attribut1 op table2.attribut2(+)...]
```